

TorQ Ops x Guard: When the Support Agent Gets the Keys

How an AI agent gets real, hands-on access to a kdb+ operational stack - and still can't take it down.

A support engineer gets a page: some tickers in the consolidated book are showing null prices. There is no button for “find the broken feed” - the work is to poke around live state, compare sources, and follow the trail to the cause. This is the kind of first-line investigation we want to hand to an AI agent, which means giving it real, hands-on access to the kdb+ estate.

A fixed menu of tools cannot anticipate every support question, so a useful agent has to be able to try its own query. But an agent free enough to investigate is also free enough to do damage: on kdb+, one careless query can take the whole process down, and a wrong or manipulated action can reach production. Telling it to “be careful” is not a control.

This brief is for teams whose support already has some hands-on access to live processes - given a handle, told to be careful, left to learn as they go. (If your agents have no real access yet, you do not have this problem.) It is how we keep that access safe: the model can propose, but Guard decides.

Guard in One Sentence

Guard is a deterministic checkpoint between an AI agent and the systems it can touch.

It works in three plain steps: the agent **proposes** an action; Guard **checks** it against a fixed policy and answers allow, ask-a-human, or block; and only an **approved** action is actually run. The agent never reaches the system directly.

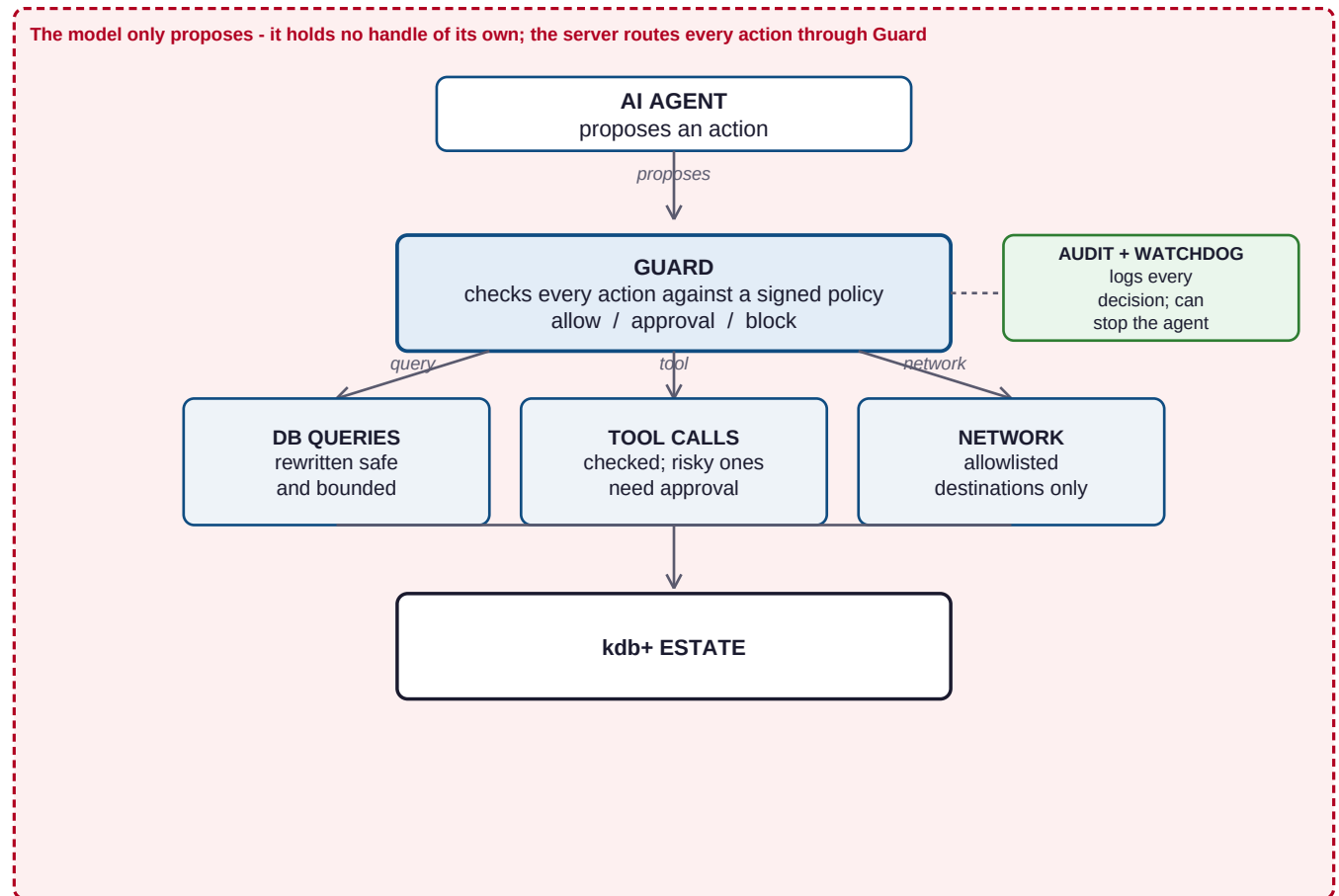
```
decision = guard.check(tool_name, tool_args, principal=user_id)
if decision.effect == "block":
    return refusal(decision)
if decision.effect == "require_approval":
    return approval_flow(decision)
return run_tool(tool_name, tool_args)
```

```
def evaluate(action):
    if policy_missing_or_invalid:
        return BLOCK
    findings = enabled_rule_packs(action)
    findings += default_deny_grants(action)
    verdict = most_severe(findings) # block > approval > allow
    audit.record(action, verdict)
    supervisor.observe(action, verdict)
    return verdict
```

A prompt cannot negotiate with Guard. The same action, plus the same policy, always gives the same result.

Core Architecture

One decision point, three controlled exits: questions to the database, actions, and anything leaving the system.

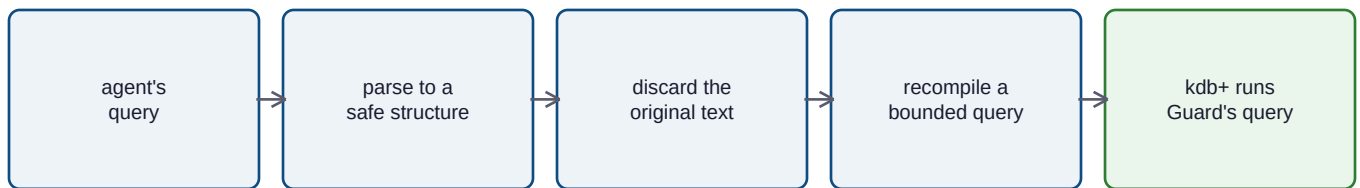


What each part is:

- **AI agent** - proposes an action; it has no authority to run anything itself.
- **Guard** - a signed-policy checkpoint that returns allow, approval or block; its decision runs in code the model never executes.
- **The three routes** - DB queries are rewritten into a bounded, allow-listed query; tool calls are permission-checked (risky ones need approval); network traffic can only reach allow-listed destinations.
- **Audit + watchdog** - every decision is logged, and a run of blocked attempts trips a breaker that halts the agent.
- **The boundary** - the model holds no database handle, shell or socket of its own; it only emits proposals, and the server runs every one through Guard first.

Query Regeneration

This is the core of Guard, and the part worth slowing down on. For database access Guard **does not run the model's query**. It reads what the query is asking for, throws the original text away, and writes a fresh, bounded query of its own.



The database receives Guard's query, never the model's original text.

A worked example. Back to that null-price page. Triaging the feed, the agent wants the average bid for NVDA and how many ticks came back null, and proposes this:

```
select avg bid, sum null bid by sym from prices_exchange where sym=`NVDA`
```

Guard never sends that string. First it **lifts** the query into a plain structured request - data, not code:

```
{ "table": "prices_exchange",
  "by": ["sym"],
  "aggs": [ {"fn": "avg", "col": "bid"},
            {"fn": "sum", "col": "bid", "of": "null", "as": "nb"} ],
  "filters": [ {"col": "sym", "op": "=", "value": "NVDA"} ] }
```

The raw text is gone - the backticks, the operators, anything executable. Then Guard **recompiles** that request into a brand-new query, stamping on the mandatory date filter and row cap the agent never wrote. This - and only this - is what kdb+ runs:

```
1000000 sublist (
  select avg bid, nb:sum null bid by sym
  from prices_exchange
  where date=2026.06.23, sym=`NVDA` )
```

The engineer still gets the number they were after - but the query is now pinned to a single date and capped, so it cannot scan the whole estate or stall the process behind the live desk.

The whole path is two calls - lift, then compile:

```
def safe_query(tool_input, principal):
    request = lift(tool_input["query"])          # parse -> structured request, or reject
    return compiler.compile(request, principal)  # rebuild a bounded, allow-listed query
```

Three stages, each closing a class of risk:

- **Parse** - pull out only the parts a query is allowed to have: table, columns, filters, time window. (The lift step above.)
- **Reject** - anything that doesn't fit that safe shape has nowhere to go. A delete, a shell call, a second statement hidden after a semicolon: none of them are in the grammar, so the request stops here, before any database sees it.
- **Compile** - write a new query from the structure, adding the row cap and date bound, and honouring the access the caller is already entitled to. kdb+ runs Guard's text, never the model's.

So the dangerous cases never reach kdb+ - they fail at parse, or at compile:

```
delete from prices_exchange
-> rejected at parse: only select / exec / meta can be expressed

select sym from trade where date=...; system "..."
-> rejected at parse: a second statement has no place in the structure

select pnl from prices_exchange where date=...
-> rejected at compile: column 'pnl' is not on the allow-list
```

The property that makes this strong: because Guard **emits** the query rather than approving the model's, a hijacked or simply mistaken model cannot get a dangerous query through - the dangerous form was never something Guard knows how to write.

What You Can Customise

Guard is driven by policy, so the same engine adapts to very different agents. You decide:

- **Tools** - which tools exist at all, which roles can call them, and which need a human to approve.
- **Data** - which tables, columns and rows are reachable, row caps, and mandatory date filters.
- **Network & process** - the platform boundary (allow-listed egress, read-only mounts, resource limits, no ambient shell or credentials) is the operator's to apply; Guard validates the deployment descriptor declares it.
- **Files** - which paths can be read or written, and which are off-limits.
- **Operations** - where a human must approve, spending ceilings, and the kill switch.
- **Audit** - how every decision is recorded, including hash-chained and off-host options.

So one engine covers a read-only reporting bot, a coding assistant locked to project files, an ops agent that needs sign-off for anything destructive, and an analyst limited to certain rows and date ranges - each just a different policy.

Why It Holds

- **Default deny:** if it isn't explicitly allowed, it doesn't happen.
- **Outside the model's reach:** the policy decision runs in code the model never executes - it only emits a proposal that Guard adjudicates.
- **No handle of its own:** the model holds no database connection, shell or socket; the server routes every action through Guard, so a careless or hijacked query is rewritten or refused before anything runs.
- **Signed policy:** the gate verifies the policy's signature against a pinned key before loading; a modified policy fails closed, so the agent cannot widen its own permissions.
- **Audit:** every decision is appended to a hash-chained log, so tampering is evident.
- **Watchdog:** a run of blocked attempts trips a circuit breaker that halts the agent until an operator resets it.
- **Deployment boundary:** read-only mounts, resource limits and allow-listed egress are the operator's platform layer - Guard validates the deployment declares them, the platform enforces them.

Conclusion

Production AI does not fail safely by default. Once an agent has credentials, tools and network access, a bad output can become a real action.

Keeping agents in bounds means changing the execution model: restrict what the agent can reach, rebuild risky outputs into safe forms, require approval for sensitive steps, and record every decision. The model can stay flexible; the environment around it must be deterministic.

The practical shift: don't rely on the AI to know its limits. Build systems where the limits are enforced before anything runs.